

08: Usage of Class.forName() in JDBC

A JDBC application loads the JDBC driver class into memory using `Class.forName()`, which then automatically registers it with the `DriverManager`. This allows the application to establish a connection with the database.

- `Class.forName()` is a static method in Java that belongs to the **Class** class.
- It is used to load a class dynamically at runtime without creating an instance of the class.

★ Use of Class class:

- The `Class` class in Java is part of the `java.lang` package and is a core part of the Java Reflection API.
- The `Class` class provides a way to access metadata about classes, interfaces, arrays, and primitive types at runtime.
- Every Java class or interface that is loaded by the JVM is represented by an instance of the `Class` class.
- Using the `Class` class, you can obtain information about a class at runtime, such as its name, constructors, methods, fields, and more.

👉 Working of Class.forName():

- o **Direct Invocation:** Since `forName()` is a static method, it can be called directly using the class name.
- o **Class Loading:** When `Class.forName()` is called with a class name as a string parameter, it loads the specified class into memory. If the class has a static block, that block is executed during the class loading.

```
Class.forName("Demo");
```

- The given example loads the `Demo` class. If `Demo` contains a static block, it will be executed as part of the class loading process.

👉 To establish a connection with the database:

The given command loads the MySQL JDBC driver class (`com.mysql.cj.jdbc.Driver`) into memory, enabling the application to connect to a MySQL database.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Alternative Method to Load the Driver:

- Instead of using `Class.forName()`, you can manually register the driver using the `DriverManager.registerDriver()` method.
- This approach explicitly registers the MySQL JDBC driver with the `DriverManager`.

```
DriverManager.registerDriver(new com.mysql.cj.jdbc.Driver());
```

★ However, starting from JDBC 4.0 and later, explicitly loading or registering the driver (either through `Class.forName()` or `registerDriver()`) is generally not necessary.

JDBC drivers are automatically loaded when an application tries to establish a connection, as long as the driver is present in the classpath. This feature, known as *automatic driver loading*, simplifies the process of working with JDBC connections.